

Lab3 报告

1 实现目标与约束

Lab3 在 Lab1 五级流水与 Lab2 真实访存的基础上，要求 CPU 正确支持条件分支与各类跳转相关控制流，并补齐一批比较、移位及 RV64 的 32 位字宽 (W) 运算。根据课程给出的目标列表，本阶段需要覆盖 B 型条件分支 (beq/bne/blt/bge/bltu/bgeu)，I 型比较与移位立即数 (slti/sltiu/slli/srli/srai)，R 型比较与移位 (sll/slt/sltu/srl/sra)，W 型移位 (slliw/srlw/sraiw/sllw/srlw/sraw)，以及 PC 相关指令 auipc、jal、jalr。实现上延续既有架构：不在整体上推翻 Lab1/Lab2 的模块划分与流水寄存器风格，只在 decode.sv、execute.sv、core.sv 等已有边界内做增量扩展；访存与写回路径保持 Lab2 已验证的语义。此外，Lab3 测试要求修改 Difftest 提交接口：对访问外设映射地址空间 (约 0x0000_0000 ~ 0x7FFF_FFFF) 的 load/store 提交跳过参考模型比对，以免 MMIO 行为无法被 NEMU 侧复现而导致误报；该 skip 必须仅在“确实是访存且地址落在外设区”时置位，否则 Difftest 会失去意义。

2 总体设计思路

整体设计仍采用“IF 取指 → ID 译码与控制 → EX 执行与分支裁决 → MEM 访存 → WB 回写”的五级结构，core.sv 中的结构化流水寄存器 if_id_t、id_ex_t、ex_mem_t、mem_wb_t 继续作为数据与控制信息的主载体。Lab3 的增量工作可以概括为三条主线：其一，在译码阶段把新指令映射到统一的 alu_op 编码，并补充 is_auipc 等控制位，使 ID/EX 能区分“普通 ALU 写回”“U 型 lui”“PC 相对 auipc”“分支/跳转”等不同语义；其二，在执行阶段扩展 ALU 与写回数据通路：比较类输出 0/1，移位类区分 64 位与 W 型 (先在 32 位上运算再按 RV64 规则符号扩展到 64 位)，auipc 写回 pc + imm_u，jal/jalr 写回当前指令的 pc+4，分支与跳转的目标 PC 仍在 EX 阶段与静态预测结果比较并产生 branch_mispredict 与 branch_correct_pc；其三，为满足 Difftest 的 skip 条件，需要在提交边界记录“本条提交是否对应访存”以及“访存物理/逻辑地址”，因此适度扩展 mem_wb 与 commit 寄存器旁路，而不改变 Lab2 MEM 级本身对总线握手与 pending 的基本策略。

分支预测仍沿用 Lab1 的静态 Always-Not-Taken 接口：predictor_static 在 ID 侧给出 pred_taken 与 pred_target，EX 根据真实 branch_taken 与 branch_target 做错预测检测；错预测时 core.sv 冲刷 IF/ID 与 ID/EX 的 valid，并由取指模块重定向到正确 PC。Lab3 新增的控制流指令仍通过同一套 is_branch/is_jal/is_jalr 与 br_funct3 进入 EX，从而避免为每种分支单独开通道，保持与 Lab1 报告中所述控制流接口一致。

3 译码阶段 (decode.sv)

译码模块在 Lab2 已支持 load/store 与 lui 的基础上，对 OP-IMM、OP、OP-IMM-32、OP-32 等 opcode 分支进行了扩展。对立即数形式，继续使用 Lab1/Lab2 已实现的 imm_i/imm_b/imm_j/imm_u 拼接规则；对 RV64I 中移位立即数的高位编码 (slli/srli/srai 对 instr[31:26] 的区分，以及 W 型 slliw 等对 funct7 的约束) 在译码里做了合法组合判断，非法编码则关闭写回，避免把未定义指令当普通算术执行。

ALU 操作类型在原有 ADD/SUB/AND/OR/XOR 之外增加了 SLL/SRL/SRA/SLT/SLTU 等编码，用于承载比较与移位。auipc 单独使用 is_auipc 标志，配合 U-type 立即数，使执行级可以用 $pc + imm_u$ 形成写回值，而不与 lui 的“直接写立即数到寄存器”混淆。分支类指令继续只置位 is_branch 并选择 imm_b，不写回寄存器；jal/jalr 保持写回使能，以便链接寄存器语义与转发路径一致。

4 执行阶段 (execute.sv)

执行级在维持 Lab2 “第二操作数来自立即数或 rs2”的前提下，将新增 ALU 操作接入统一的 alu_raw 计算。比较指令通过有符号/无符号比较得到一位结果并零扩展到 64 位；移位指令根据 is_word 区分 64 位移位与 W 型移位：W 型在 $op1[31:0]$ 上完成移位，再交给后续字宽符号扩展路径处理。分支比较仍使用 $op1$ 与 $rs2_val$ ，与立即数数据通路解耦，避免 Lab2 中为访存地址计算引入的 $op1+imm$ 路径干扰分支判断。

控制流方面，jal 的目标为 $pc + imm_j$ ，jalr 的目标为 $(rs1 + imm_i) \& \sim 1$ ，分支目标为 $pc + imm_b$ ；写回值对 jal/jalr 统一为 $pc+4$ ，auipc 为 $pc+imm_u$ ，lui 仍为写回 imm_u 本身。错预测条件继续是“是否采用分支/跳转”与“预测是否一致、若采用则目标是否一致”的组合，与 Lab1 中 EX 级与 IF 重定向的接口一致。为便于阅读，在 jal/jalr 目标计算处保留了与 RISC-V 规范对应的简短注释，减少后续上板或波形调试时的理解成本。

5 顶层与 Dfftest (core.sv)

id_ex_t 增加 is_auipc 字段，使执行级可以独立识别 auipc 而不侵占 lui 语义。为满足 Lab3 文档中对 DfftestInstrCommit.skip 的要求，mem_wb_t 增加了 is_load、is_store 以及 mem_addr 字段，用于在 WB→commit 打一拍的路径上保留“本条指令是否为访存指令及其地址”。commit 寄存器侧相应记录 commit_is_mem 与 commit_mem_addr，并将 .skip 连接为：仅当提交对应访存且地址的最高位为 0（落在课设约定的外设映射低端区间）时跳过比对，这与文档给出的 $(mem \&\& memaddr[31]==0)$ 含义一致，同时避免对所有指令恒跳过。

流水控制仍区分 mem_wait 与 branch_mispredict、load-use 冒泡等；Lab2 的“访存未完成则冻结前级、当拍不推进 mem_wb”策略保留，以保证总线事务与提交顺序一致。

6 访存级 (mem.sv)

MEM 级在 Lab2 的 pending/等待模型上未做结构性重写，仅增加向顶层输出的 out_is_load、out_is_store 与 out_mem_addr。这些信息用于 WB/commit 侧的 Dfftest skip 判定，不改变 load/store 的请求条件、dresp.data_ok 完成条件以及 pending 清除逻辑本身；thus Lab2 访存语义保留。

7 取指级与系统集成中的交互 (fetch.sv)

Lab2 引入 mem_wait 后，core.sv 将 stop_fetch 置为 $cpu_halt \mid mem_wait$ ，用于在访存未完成时暂停取指前进，以避免流水线前排继续涌入指令。原先取指逻辑在“停顿状态下仍收到指令返回 iresp.data_ok”时，有可能错误地将内部请求状态清零，导致此后取指请求永

远无法再次拉高，表现为长时间无任何指令提交。修正策略是：在停顿且收到返回时，保持请求有效并保持当前 PC，不在该边界丢失 pending，从而在访存完成后仍能恢复顺序取指。

```
[src/device/io/mmio.c:19,add_mmio_map] Add mmio map 'clint' at [0x38000000, 0x3800ffff]
[src/device/io/mmio.c:19,add_mmio_map] Add mmio map 'uartlite' at [0x40600000, 0x4060000c]
[src/device/io/mmio.c:19,add_mmio_map] Add mmio map 'uartlite1' at [0x23333000, 0x2333300f]
The first instruction of core 0 has committed. Difftest enabled.
[WARNING] difftest store queue overflow
AES benchmark + correctness
Running AES correctness checks...
[TEST] AES-128 test vector Encrypt
[PASS] AES-128 test vector Encrypt passed
[TEST] AES-128 test vector Decrypt
[PASS] AES-128 test vector Decrypt passed
[PASS] AES correctness checks passed
[RNG] heavy_rng_mod result=0, time(us)=104151
[RNG] heavy_rng_mod it/s=9615
Core 0: HIT GOOD TRAP at pc = 0x80000030
total guest instructions = 1,243,595
instrCnt = 1,243,595, cycleCnt = 3,051,479, IPC = 0.407538
Seed=0 Guest cycle spent: 3,051,480 (this will be different from cycleCnt if emu loads a snapshot)
Host time spent: 15,630ms
This emulator compiled with JTAG Remote Bitbang client. To enable, use +jtag_rbb_enable=1.
Listening on port 23334
```

串口调试助手 + TCP/UDP

AA

?

⚙

端口名：

COM3

波特率：

9600

数据位：

8

校验位：

None

停止位：

1

RI

DSR

CTS

DTR

RTS

关闭

接收设置：

☆

将接收保存到文件

十六进制显示

自动断帧

脚本

20

ms

?

Add Timestamp

保存数据

清空数据

发送设置：

发送文件

十六进制发送

脚本

定时发送

显示发送字符串

终端模式

自动开关串口

✓

自动重连

扩展命令

ADD8

1.0

秒

AES benchmark + correctness

Running AES correctness checks...

[TEST] AES-128 test vector Encrypt

[PASS] AES-128 test vector Encrypt passed

[TEST] AES-128 test vector Decrypt

[PASS] AES-128 test vector Decrypt passed

[PASS] AES correctness checks passed

[RNG] heavy_rng_mod result=0, time(us)=2030944

[RNG] heavy_rng_mod it/s=493

发送：0

接收：329 - 76

复位计数